

Ejercicio 6

Investigación REACT

¿Qué son los Hooks de Rendimiento ?

Los Hooks de rendimiento son Hooks de React que ayudan a optimizar una aplicación evitando trabajo innecesario: cálculos repetidos y renderizados que no hacen falta. Los principales son **useMemo**, **useCallback** y **useTransition**. Sirven para que la interfaz siga siendo fluida cuando los componentes hacen cálculos pesados o se renderizan muchas veces.

¿Cuáles son sus características y para qué sirve?

- ☆ **useMemo:** guarda (memoriza) el resultado de un cálculo y solo lo vuelve a hacer si cambian sus dependencias.
- ☆ **useCallback:** guarda una función para que no se cree una nueva en cada renderizado. Es útil al pasarla como prop a componentes hijos.
- ☆ **useTransition:** marca una actualización como no urgente, para que la interfaz no se trabe mientras se procesa.
- ☆ Reciben un arreglo de dependencias que indica cuándo deben recalcularse.
- ☆ Se usan solo cuando hay un problema real de rendimiento: usarlos en todos lados agrega complejidad sin beneficio.
- ☆ Suelen combinarse con **React.memo**, que evita que un componente se vuelva a renderizar si sus props no cambiaron.

Ejemplo de implementación:

```

1  import { useState, useMemo } from "react";
2
3  function calcularFactorial(n) {
4    console.log("Calculando factorial...");
5    let resultado = 1;
6    for (let i = 2; i <= n; i++) {
7      resultado *= i;
8    }
9    return resultado;
10 }
11
12 function Factorial() {
13   const [numero, setNumero] = useState(5);
14   const [contador, setContador] = useState(0);
15
16   const factorial = useMemo(() => calcularFactorial(numero), [numero]);
17
18   return (
19     <div>
20       <p>Factorial de {numero}: {factorial}</p>
21       <button onClick={() => setNumero(numero + 1)}>Aumentar número</button>
22
23       <p>Contador: {contador}</p>
24       <button onClick={() => setContador(contador + 1)}>Sumar contador</button>
25     </div>
26   );
27 }
28
29 export default Factorial;
30

```

Explicación: el factorial se calcula dentro de **useMemo**, con **numero** como dependencia. Al presionar "Aumentar número", cambia la dependencia y el cálculo se rehace. Al presionar "Sumar contador", el componente se renderiza de nuevo, pero como **numero** no cambió, **useMemo** devuelve el valor guardado y no se vuelve a calcular (en la consola se ve que "Calculando factorial..." no aparece otra vez).

¿Qué es el contexto en React ?

El contexto (*Context*) es un mecanismo de React para compartir datos entre muchos componentes sin tener que pasarlos manualmente por props en cada nivel del árbol. Sirve para información global o que necesitan muchos componentes, como el tema (claro/oscuro), el idioma o el usuario logueado. Evita el problema conocido como *prop drilling* (pasar props a través de componentes intermedios que no las usan).

¿Cuáles son sus características y para qué sirve?

- ☆ Se crea con **createContext()**.
- ☆ Un componente **Provider** envuelve a los componentes que van a tener acceso al dato y le da su valor (**value**).

- ☆ Los componentes que necesitan el dato lo leen con el Hook **useContext**.
- ☆ Cuando el valor del contexto cambia, se vuelven a renderizar los componentes que lo consumen.
- ☆ Es ideal para datos globales, no para cualquier dato: para datos que usan pocos componentes conviene seguir usando props.

¿Qué diferencia existen con los props?

- ☆ **Props:** los datos se pasan de padre a hijo de forma explícita, nivel por nivel. Son ideales para datos específicos de un componente.
- ☆ **Contexto:** los datos quedan disponibles para cualquier componente dentro del Provider, sin importar qué tan profundo esté. Es ideal para datos globales.

Ejemplo de implementación:

```
1  import { createContext, useContext } from "react";
2
3  const UsuarioContext = createContext();
4
5  function Perfil() {
6    const usuario = useContext(UsuarioContext);
7    return <h2>Bienvenida, {usuario}</h2>;
8  }
9
10 function Panel() {
11   return <Perfil />;
12 }
13
14 function App() {
15   return (
16     <UsuarioContext.Provider value="Jane">
17       <Panel />
18     </UsuarioContext.Provider>
19   );
20 }
21
22 export default App;
23
```

Explicación: createContext() crea el contexto UsuarioContext. En App, el Provider envuelve a Panel y le da el valor "Jane". Panel no recibe ni pasa ningún prop, pero Perfil, que está más adentro, obtiene el valor directamente con useContext y muestra "Bienvenida, Jane". Con props, habría que pasar el dato de App a Panel y de Panel a Perfil.